

Mesh Agent System:
Architecture, Control, and Memory
Technical Report

Alan Kuhnle

May 2026

Abstract

This document describes the architecture of the `hello-world` mesh agent system—a multi-agent coordination framework built around heterogeneous LLM backends, a plan-and-execute controller, and an episodic memory system. The report covers five layers: LLM backend dispatch, tool routing through `complete_with_tools`, the plan-and-execute controller with its native multi-turn executor loop, security hardening of subprocess and credential handling, and the episodic memory system (formation, curation, and retrieval). Configuration lives in `mesh.yaml`; runtime dispatch lives in `mesh/llm.py`; the controller lives in `mesh/harness/loop.py`; memory state lives in `~/.mesh/memory/`.

Contents

Part I

System Architecture

Chapter 1

Introduction

The mesh system is a process-based multi-agent coordination framework where users and AI agents communicate through a central message router. Unlike monolithic agent architectures that pair a single LLM with a fixed tool set, the mesh is designed around three principles:

1. **Heterogeneous backends.** Each agent can use a different LLM backend—OpenAI, Anthropic, Claude Code, Z.ai, Codex, or a standalone harness subprocess—selected per-agent via configuration. This enables cost/capability tradeoffs: a cheap local model for routine tasks, an expensive API model for hard problems, and a dedicated assessor LLM for orchestration.
2. **Shared tool surface.** All agents access the same tools—email, calendar, notes, web search, literature, memory, finance, and inter-agent communication—through a unified `mesh-tool` CLI. Tool routing adapts to each backend’s capabilities: native function calling for OpenAI/Anthropic, shell invocation for Claude Code/Codex, and structured XML for legacy paths.
3. **Persistent, project-organized memory.** Agents build episodic memory over their lifetimes through an automated formation pipeline. Memories are organized by project, curated into living project maps, and surfaced via a two-stage table-of-contents pipeline that balances relevance and diversity.

The system has been in continuous operation since January 2026, running 9 agents across multiple hosts. This report documents the architecture as implemented, including the plan-and-execute controller that orchestrates multi-phase code editing tasks, the native multi-turn executor loop that eliminates redundant file reads across phase boundaries, and the benchmark infrastructure used to evaluate configurations.

Chapter 2

Getting Started

This chapter walks through setting up a minimal mesh instance from scratch: one router, one agent, and a terminal interface.

2.1 Prerequisites

- **Python 3.11+** with `pip` and `venv`.
- An **OpenAI-compatible API key** (for the default backend). For Claude Code backends, a Claude Max subscription and the `claude` CLI binary are required instead.
- **Linux** is the primary development platform. macOS works for the core system; Windows is untested.
- **Git** for cloning the repository.

2.2 Installation

```
git clone <repo-url> hello-world
cd hello-world
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

The `mesh-tool` CLI lives at the repository root and is automatically added to `PATH` via agent prompt injection. For manual use, run it directly:

```
./mesh-tool                # list all available tools
./mesh-tool gmail_list_recent # show usage for a specific tool
```

2.3 Minimal Configuration

All configuration lives in `mesh.yaml` at the repository root. The production deployment defines 46 backends and 9 agents, but a minimal setup needs only the router section and one backend.

Environment variables. Create an `env.bash` file exporting the API keys referenced by `mesh.yaml`. At minimum:

```
# env.bash      DO NOT commit this file
export OPENAI_API_KEY="sk-..."
export MESH_AUTH_TOKEN="$(openssl rand -hex 32)"
```

The `MESH_AUTH_TOKEN` is a shared secret used by agents to authenticate with the router. For a local-only setup, any random hex string works.

Minimal mesh.yaml. A working single-backend configuration:

```
router:
  host: 127.0.0.1
  port: 7700
  ws_port: 8765
  storage_path: ~/log/chats/mesh-storage/messages.db
  auth_enabled: true
  auth_mode: per_user

llm_backends:
  default:
    backend_type: openai
    api_key: ${OPENAI_API_KEY}
    base_url: https://api.openai.com/v1
    default_model: gpt-4o
    max_tokens: 4096
    temperature: 0.7

nodes:
  agent:researcher:
    llm_backend: default
    system_prompt_file: researcher
```

The `backend_type` field determines which dispatch method is used (see Chapter ??). The `nodes` section maps agent identities to backends and prompt files.

2.4 Quick Start

Three terminal windows are needed: one for the router, one for an agent, and one for the user interface.

Terminal 1 — Router. The router is the central message broker. It classifies incoming messages, manages agent connections, and persists conversation history.

```
source env.bash
python run_router.py
```

Terminal 2 — Agent. Each agent connects to the router via TCP, registers its identity, and waits for dispatched work. The `--agent` flag selects the agent type (which determines the system prompt); `--nickname` sets the display name.

```
source env.bash
python run_agent.py --agent researcher --nickname alice \
    --auth-token $MESH_AUTH_TOKEN
```

Terminal 3 — User TUI. The terminal user interface connects to the router, displays a unified timeline of messages from all agents, and supports target selection, markdown rendering, and session persistence.

```
source env.bash
python run_user_tui.py --nickname yourname
```

On startup the router will prompt for user creation if `auth_mode: per_user` is set. The TUI supports `/target <name>` to set a default recipient and `@<nickname>` mentions to address specific agents.

2.5 Verifying It Works

1. The router terminal should show `[INFO] Router started on 127.0.0.1:7700`.
2. The agent terminal should show `[INFO] Registered as agent:researcher:alice`.
3. The TUI should display a connected status and any presence notifications (e.g., “alice joined”).
4. Type a message in the TUI. The router classifies it and dispatches it to alice, who responds via the configured LLM backend.

For a dashboard view of all connected agents, run:

```
mesh-tool mesh_status
```

2.6 Adding More Agents

To add a second agent, start another `run_agent.py` process with a different `--agent` type and `--nickname`:

```
source env.bash
python run_agent.py --agent sysadmin --nickname bob \
    --auth-token $MESH_AUTH_TOKEN
```

Multiple agents can share the same backend or use different ones. To configure per-agent backends, add entries to the `nodes` section of `mesh.yaml` keyed by the full node ID (e.g., `agent:sysadmin:bob`). See Chapter ?? for backend configuration details.

2.7 Notes for Public Release

The production `mesh.yaml` contains 46 backends with paths hardcoded to absolute paths and API keys referenced via environment variables. To prepare the repository for public release:

- Create `env.bash.example` listing required variable names without values.

- Create `mesh.yaml.example` with the minimal configuration shown above (one backend, no hardcoded paths).
- Add `.gitignore` entries for `env.bash`, `firebase-credentials.json`, and any token files.
- Replace hardcoded absolute paths with `$HOME` expansion or relative paths.
- Write a `QUICKSTART.md` linking to this chapter for users who prefer plain-text instructions over the PDF.

Chapter 3

System Overview

The mesh is a multi-agent system where a central **router** classifies user messages and either responds directly or dispatches work to agent nodes. The router has its own LLM backend (`router_v2_llm_backend`) and runs a state machine (IDLE / BUSY) that is distinct from any worker LLM.

Each agent node has:

- An **LLM backend** (how it calls language models),
- A **tool surface** (what tools are available),
- An optional **harness layer** (plan-and-execute controller).

Router states. RouterV2 operates a two-state machine:

IDLE Incoming message → LLM classification call → `{needs_response, needs_worker}`. If `needs_worker` is false, the router generates a direct response and stays IDLE. If true, the router sends an immediate acknowledgment, injects relevant memories, and spawns a worker.

BUSY Worker is running. New messages are handled by the router’s own LLM, which peeks at the worker’s in-progress context to generate contextual interim responses.

Unified harness-backend dispatch. When the router’s own LLM backend is itself a harness-type backend (Claude Code, Codex, or `mesh-harness`—enumerated in `HARNESS_BACKENDS` in `router_v2.py`), the router cannot use native tool calls to launch workers. Instead, these backends share a unified dispatch contract: router-only tools (`worker_launch`, `worker_status`) are stripped from the effective tool catalog, the router receives exactly one outer iteration, and the LLM must emit a `<dispatch_worker>` XML block in its text response to trigger worker creation. Non-harness backends (e.g., DeepSeek direct API) retain the native tool-call dispatch path unchanged.

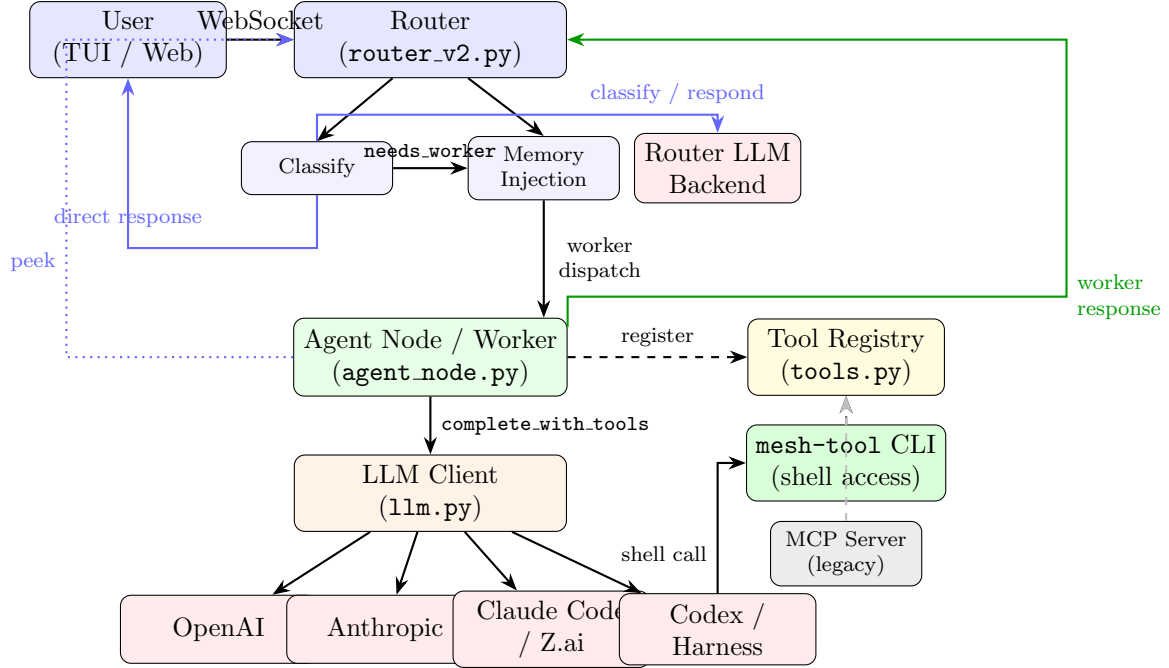


Figure 3.1: Message flow through the mesh. The router has its own LLM backend (`router_v2_llm_backend`, separate from the worker's) used for classification and direct response generation (blue arrows). On `needs_worker`, the router injects relevant memories and dispatches to the agent node, which calls its own LLM backend via `complete_with_tools`. While a worker is running (BUSY), the router generates interim responses via worker peek (dotted blue line). When the worker completes, the agent node sends the response back through the router to the user (green arrow, right side). Codex / Harness subprocesses access mesh tools via the `mesh-tool CLI`. Dashed lines indicate tool discovery; gray indicates legacy paths.

Chapter 4

LLM Backend Dispatch

Backends are configured in `mesh.yaml` under `llm_backends:` and assigned to agents via the `llm_backend` field on each node. The `LLMClient` class in `mesh/llm.py` dispatches to a backend-specific completion method based on `LLMConfig.backend`.

4.1 Backend Types

4.2 Live Backend Examples

The production `mesh.yaml` defines 46 backend configurations. Representative examples:

4.3 Configuration Reference

4.3.1 Backend Configuration (`mesh.yaml`)

Each backend is defined under `llm_backends:` with at minimum:

```
llm_backends:
  my-backend:
    backend_type: openai           # openai | anthropic | claude-code
                                   # | zai | codex | mesh-harness
    api_key: ${ENV_VAR}
    base_url: https://...         # optional override
    default_model: gpt-5.1
    max_tokens: 16384
    temperature: 0.7
```

4.3.2 Agent Node Configuration

Each agent node under `nodes:` references a backend and specifies tool access:

```
nodes:
  agent:sysadmin:bob:
    nickname: bob
    llm_backend: claude-code-opus
    system_prompt_file: sysadmin.md
    tools: *common_tools          # YAML anchor
```

Type	Method	Notes
openai	<code>_complete_openai</code> / <code>_complete_openai_responses</code>	Two paths: Chat Completions API for standard models; Responses API for reasoning models (detected via <code>should_use_responses_api</code>). Supports native function calling.
anthropic	<code>_complete_anthropic</code>	Claude API with native <code>tool_use</code> blocks. Supports vision (image extraction from history).
claude-code	<code>_complete_claude_code</code>	Claude Code CLI subprocess. Supports MCP sidecar for mesh tools. Multi-account fallback via <code>cc_fallback_homes</code> .
zai	<code>_complete_zai</code>	Z.ai proxy. Same interface as <code>claude-code</code> (MCP sidecar, CC CLI subprocess).
codex	<code>_complete_codex</code>	Codex CLI subprocess with its own built-in tool loop (<code>codex:shell</code> , <code>codex:patch</code> , etc.). Mesh tools available via <code>mesh-tool</code> CLI on <code>PATH</code> (repo root prepended to subprocess env).
mesh-harness	<code>_complete_mesh_harness</code>	Standalone TAOR (think-act-observe-reflect) loop as a subprocess. Usable as any agent backend, not only for plan-and-execute. Tool set is configurable via <code>harness_toolset</code> (see Chapter ??).

Table 4.1: LLM backend types and their dispatch methods.

```
memory_enabled: true
memory_llm_backend: deepseek-direct
memory_formation_v3_enabled: true
memory_retrieval_redesign_enabled: true
```

4.3.3 Specialized LLM Clients

Agents may use multiple LLM backends simultaneously:

- **Worker backend** (`llm_backend`) — primary LLM for conversation and tool use.
- **Router V2 backend** (`router_v2_llm_backend`) — separate LLM for the routing/classification layer.
- **Memory formation backend** (`memory_llm_backend`) — dedicated LLM for the v3 segmenter, decoupled from the worker backend (e.g., `deepseek-direct` while the worker uses `claude-code-opus`).
- **Assessor backend** (in plan-and-execute) — the controller/checkpoint LLM, configured per `mesh-harness` backend definition.

Name	Type	Model	Use Case
default	openai	gpt-5.1	General agents
openai-reasoning-*	openai	o4-mini / o3	Reasoning tasks
claude-code-opus	claude-code	opus	Most agents
deepseek-direct	openai	deepseek-v4-pro	Memory formation
vega-qwen36	openai	local-27b	Local executor (Qwen3.6-27B)
codex-5.5	codex	gpt-5.5	Codex agents
mesh-harness-qwen36-pe	mesh-harness	Qwen3.6-27B + DS-pro	Plan-and-execute

Table 4.2: Selected backend configurations from production `mesh.yaml`.

Chapter 5

Tool System

5.1 Tool Routing: `complete_with_tools`

The central dispatch method is `LLMClient.complete_with_tools()` in `mesh/llm.py`. It accepts a `ToolRegistry`, a list of enabled tool names, and optional MCP configuration, then routes based on backend type.

5.1.1 Dispatch Paths

Path	Condition	Tool Format	Response Parsing
CC briefing	<code>cc_system_prompt</code> & <code>cc_user_prompt</code> set, <code>backend</code> $\in$ {cc, zai}	None (worker dispatch)	Plain text, no tool calls
OpenAI native	<code>backend</code> == "openai" & tools	<code>get_openai_tools()</code> or <code>get_openai_responses_tools()</code>	<code>_convert_openai_tool_calls()</code>
Anthropic native	<code>backend</code> == "anthropic" & tools	<code>get_anthropic_tools()</code>	<code>_convert_openai_tool_calls()</code>
CC/zai + MCP	<code>backend</code> $\in$ {cc, zai} & <code>mcp_config</code> [Legacy]	MCP sidecar (tools internal to CC)	Plain text; CC executes tools internally. Being replaced by <code>mesh-tool CLI</code> .
XML fall-through	Codex, harness, CC/zai without MCP [Legacy]	<code>format_tools_prompt()</code> → XML <code><mesh_call></code> in prompt	<code>parse_tool_calls()</code> regex on response. Emits <code>DeprecationWarning</code> .

Table 5.1: Tool routing paths in `complete_with_tools`.

5.1.2 Tool Format Conversion

The `ToolRegistry` (in `mesh/tools.py`) provides format converters for each backend:

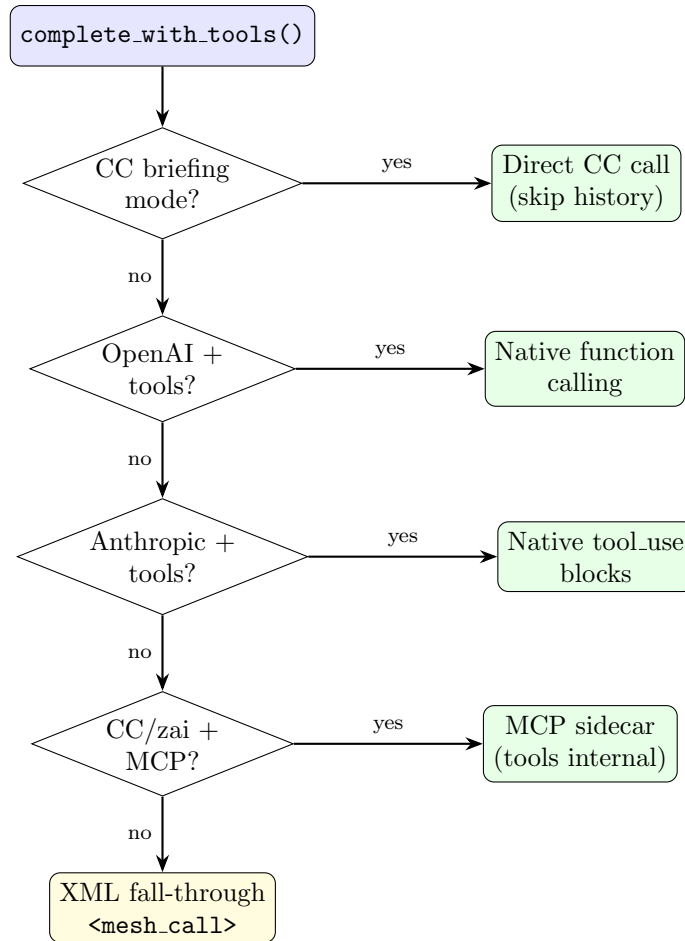


Figure 5.1: Decision tree in `complete_with_tools`. The first matching branch is taken; unmatched backends fall through to XML.

- `get_openai_tools()` — OpenAI Chat Completions function calling format.
- `get_openai_responses_tools()` — OpenAI Responses API flattened tool format.
- `get_anthropic_tools()` — Anthropic `tool_use` format.
- `format_tools_prompt()` — XML `<mesh.call>` prompt block for completion-only backends.

5.2 mesh-tool CLI

The `mesh-tool` CLI provides shell-based access to mesh tools, replacing the MCP sidecar for all backends with shell access. Any agent that can execute shell commands can call mesh tools directly—no schema injection, XML parsing, or MCP plumbing required.

Architecture. `mesh-tool` is a bash wrapper at the repository root that activates the virtualenv and runs `python -m mesh.cli.mesh_tool`. It imports the same `tool_implementations.py` functions used by the `ToolRegistry`—no reimplementations.

```

mesh-tool (bash wrapper, repo root)
-> python -m mesh.cli.mesh_tool

```



```

-> imports mesh.tool_implementations
-> looks up tool in ToolRegistry
-> parses --key value args (type coercion)
-> executes handler (async-aware)
-> JSON to stdout, errors to stderr

```

Invocation contract.

- `mesh-tool` (no args) — lists all available tools grouped by category.
- `mesh-tool <name>` (no required args provided) — prints usage with required/optional parameters.
- `mesh-tool <name> --help` — same as above.
- `mesh-tool <name> --arg value ...` — executes the tool.
- Wrong/missing arguments — prints error to stderr with “Did you mean?” suggestions, usage to stdout, exit 1.

Output contract.

- **Success:** JSON to stdout, exit 0.
- **Error:** human-readable message to stderr, usage to stdout, exit 1.
- **Help:** usage text to stdout, exit 0.

Identity. The environment variable `MESH_NODE_ID` identifies the calling agent (e.g., `agent:sysadmin:bob`). It is propagated through two layers: (1) `os.environ` is set at agent construction for tool subprocess inheritance (`bash_exec`, etc.); (2) `LLMConfig.node_id` is threaded explicitly into each LLM subprocess environment (`_run_cc_subprocess`, `_complete_codex`, `_complete_mesh_harness`). The explicit path ensures correctness if multiple agents share a process.

PATH wiring. The agent runtime prepends the repository root to `PATH` in both `_run_cc_subprocess()` and `_complete_codex()`, making `mesh-tool` discoverable by any subprocess spawned by these backends.

Prompt injection. All agent prompts auto-include `mesh/prompts/mesh_tools.md` via the `load_prompt_file()` auto-include mechanism (same pattern as `channel_policy.md` and `memory.md`). This gives agents concise instructions on discovering and calling `mesh-tool` from the shell.

Tool surface. 53 tools across Gmail, web search, notes, literature, calendar, memory, maps, mesh info, finance, and utility categories. Plus `send_message` for agent communication.

Agent-routed tools. Tools requiring agent-local state (`send_message`, `channel_list`, `mesh_status`, `agent_status`, `schedule_wake`, `gmail_send_message`, `gmail_reply_to`, `account_set_current`, etc.) are dispatched via the agent’s Unix socket at `~/.mesh/sockets/{sanitized_id}.sock` (directory mode 0700, owner-only access). The CLI falls back to the legacy `/tmp/mesh_agent_*` path for agents started before the hardening change. Memory tools also prefer the socket path when available.

Gmail account switching. All Gmail tools accept an `--account` flag for stateless per-call account selection (e.g., `mesh-tool gmail_list_recent --account personal --limit 5`). Each CLI invocation is a fresh subprocess; there is no persistent account state between calls. Write tools

(`gmail_send_message`, `gmail_reply_to`) are routed through the agent socket with `skip_confirmation=True`; the socket handler saves the current account, switches to the requested account before executing, and restores in a `try/finally` block.

Stdin mode (shell interpolation hardening). When a parameter value is passed as `-` and `stdin` is not a TTY, `mesh-tool` reads the value from standard input instead of from the command-line argument. This enables single-quoted heredocs to bypass bash expansion of `$`, backticks, and `$(...)` constructs:

```
mesh-tool gmail_send_message --to "x@y.com" --subject "Invoice" --body -
<<'EOF'
The payment of $550 is still outstanding.
EOF
```

Without this mechanism, `$550` would be interpolated as the shell variable `$5` concatenated with `50`, producing a mangled message. The fix is a three-layer approach: (1) `stdin` mode in the CLI parser (`mesh_tool.py`), (2) shell-safety instruction sections in agent prompts documenting the `<<'EOF'` convention, and (3) the documented convention in `mesh/prompts/mesh_tools.md`.

Map tool socket routing. The tools `map_edit`, `map_create`, `map_review`, and `set_project_context` are routed through the agent socket via a prefix-matching check in `_execute()` (not via `AGENT_ROUTED_TOOLS`, which is reserved for placeholder-handler tools). This ensures that worker subprocesses can modify project maps via the parent agent’s initialized memory system rather than attempting to initialize their own (which would fail with “memory not initialized” errors).

`gmail_list_recent` / `gmail_list_unread`. Two convenience tools for inbox browsing: `gmail_list_recent` returns the N most recent emails (newest first, no query required), and `gmail_list_unread` returns unread emails only. Both accept the `--account` flag and are primarily used by assistant agents for unknown-target email retrieval.

5.3 MCP Bridge (Legacy)

Status: Being replaced by `mesh-tool` CLI (§??). The MCP sidecar is still functional but is no longer the recommended path. New agents should use `mesh-tool` via shell instead.

For `claude-code` and `zai` backends, mesh tools were historically exposed via an MCP (Model Context Protocol) server sidecar defined in `mesh/mcp_server.py`. The CC subprocess is started with `--mcp-config` pointing to a temporary JSON file that configures the MCP server.

Agent-local tools. A subset of tools require agent-local state (WebSocket connection, scheduler, etc.) and are routed to `agent_node.py` via Unix socket:

```
send_message, channel_list, channel_members, schedule_wake, schedule_list, schedule_cancel,
agent_shutdown, mesh_status, agent_status
```

5.4 Tool Surface

5.4.1 Mesh Tools

Mesh tools are defined in `mesh/tool_implementations.py` and registered via `mesh/clients/account_manager.py` (the `ToolHost`). They cover external services:

Category	Tools
Email	<code>gmail_list_from_date</code> , <code>gmail_get_email</code> , <code>gmail_send_message</code> ^S , <code>gmail_reply_to</code> ^S , <code>gmail_search_emails</code> , <code>gmail_list_recent</code> , <code>gmail_list_unread</code>
Calendar	<code>calendar_list_on_date</code> , <code>calendar_create_event</code> ^C , <code>calendar_delete_event</code> ^C
Notes	<code>notes_search</code> , <code>notes_list</code> , <code>notes_get</code> , <code>notes_read</code> , <code>notes_add</code> , <code>notes_delete</code> , <code>notes_edit</code>
Web	<code>exa_search</code> , <code>exa_fetch_full</code> , <code>extract_url</code>
Literature	<code>literature_search</code> , <code>literature_fulltext</code> , <code>arxiv_{search,get,fulltext}</code> , <code>pubmed_{search,get,fulltext,related}</code>
Memory	<code>memory_{search,get,list,add,delete}</code> , <code>map_{get,list,edit,create,review}</code> , <code>set_project_context</code> , <code>remember</code>
Finance	<code>plaid_{link_start,link_status,accounts,sync,</code> <code>transactions,unlink}</code>
Mesh	<code>send_message</code> , <code>mesh_list</code> , <code>mesh_status</code> , <code>agent_status</code> , <code>agent_shutdown</code> , <code>channel_{list,members}</code>
Scheduling	<code>schedule_{wake,list,cancel}</code>
Browser	<code>browser_{session.*,goto,get_url,snapshot_controls,</code> <code>read_text,click,fill,type,press,select,back}</code>
Utility	<code>current_time</code> , <code>tool_help</code> , <code>echo</code> , <code>account_{get_current,list,set_current}</code> , <code>personality_{get,set}</code> , <code>synthetic_quota</code> , <code>claude_code_usage</code>

Table 5.2: Mesh tools by category. ^S = socket-routed (via agent Unix socket with `skip_confirmation`). ^C = confirmation-gated (requires user approval).

5.4.2 Harness Tools

The harness layer (`mesh/harness/tools/__init__.py`) defines three tool sets that mirror or extend the Codex tool surface:

5.5 Backend–Tool Compatibility Matrix

Set	Tools	Purpose
HARNESS_TOOLS (8)	<code>apply_patch</code> , <code>shell</code> , <code>list_dir</code> , <code>file_read</code> , <code>file_edit</code> , <code>phase_complete</code> , <code>grep</code> , <code>find_files</code>	Full executor surface. Registered with the global <code>ToolRegistry</code> on import.
PLANNER_TOOLS (4)	<code>file_read</code> , <code>list_dir</code> , <code>grep</code> , <code>find_files</code>	Read-only subset for the observe phase.
MESH_READ_ONLY_TOOLS (45)	All read-only mesh tools (search, list, get variants; no write/confirmation tools)	Safe-to-call during observe phase. Filtered from agent’s full tool list.

Table 5.3: Harness tool sets.

Backend	Native	XML	MCP	mesh-tool	Mesh Tools	Harness
openai	✓				✓	via harness
anthropic	✓				✓	via harness
claude-code			†	✓	✓	via harness
zai			†	✓	✓	via harness
codex				✓	✓	built-in
mesh-harness		†		✓	✓	✓

Table 5.4: Backend–tool compatibility. “Native” = API-level function calling. “XML” = `<mesh.call>` in prompt. “MCP” = tools via MCP sidecar. “mesh-tool” = tools via shell CLI. † = deprecated path (still functional but being phased out). “via harness” = available when the backend runs inside the harness layer. **Key change:** codex now has mesh tool access via `mesh-tool` CLI.

Part II

Execution and Control

Chapter 6

The Mesh Harness

The mesh harness is a standalone TAOR (Think-Act-Observe-Repeat) execution loop implemented in `mesh/harness/loop.py`. It can operate as a simple iterative executor (standard mode) or as the substrate for the plan-and-execute controller (Chapter ??).

6.1 Overview

All harness logic resides in a single file: `mesh/harness/loop.py`. Supporting definitions are in `mesh/harness/tools/__init__.py` (tool sets), `mesh/harness/protocol.py` (event emission), and `mesh/config.py` (configuration plumbing).

The entry point is `run_loop()`, which accepts an LLM client, conversation history, tool names, and configuration parameters. In standard mode, it runs a simple loop: call the LLM, execute any tool calls, append results to history, repeat until the LLM produces a final response or hits the iteration limit.

6.2 Harness Toolset Configuration

The `mesh-harness` backend controls which tools the subprocess receives via two config fields on `LLMConfig`:

- `harness_toolset` (default: "legacy") — "legacy" exposes the full mesh tool set (email, calendar, notes, memory, browser, etc. via the parent agent's `ToolRegistry`); "harness" restricts to the 4-tool Codex-style surface (`file_read`, `list_dir`, `grep`, `find_files` plus `apply_patch`, `shell`).
- `harness_tools` (default: "") — comma-separated tool names. When non-empty, *overrides* `harness_toolset` entirely, giving fine-grained control.

The toolset flag is passed to the subprocess via `--toolset` or `--tools` CLI arguments (see `_complete_mesh_harness` in `llm.py`).

Codex as direct backend vs. through harness. There is an important distinction between two ways an agent can use Codex:

1. **Direct backend** (`backend_type: codex`) — `_complete_codex` launches the Codex CLI subprocess with its own built-in tools (`codex:shell`, `codex:patch`, etc.). Mesh tools are now available via `mesh-tool` CLI on `PATH` (repo root prepended to subprocess environment).
2. **Through harness** (`backend_type: mesh-harness` with a codex sub-backend) — the harness wraps the Codex model in its own TAOR loop and can expose mesh tools via `harness_toolset`:

legacy. In this mode, mesh tools are available to the harness loop but the inner model does not see them directly.

6.3 Tool Sets

Defined in `mesh/harness/tools/__init__.py`:

Set	Tools
HARNESS_TOOLS	<code>apply_patch</code> , <code>shell</code> , <code>list_dir</code> , <code>file_read</code> , <code>file_edit</code> , <code>phase_complete</code> , <code>grep</code> , <code>find_files</code>
PLANNER_TOOLS	<code>file_read</code> , <code>list_dir</code> , <code>grep</code> , <code>find_files</code>
MESH_READ_ONLY_TOOLS	45 tools: <code>gmail_*</code> , <code>exa_*</code> , <code>notes_*</code> , <code>arxiv_*</code> , <code>pubmed_*</code> , <code>calendar_*</code> , <code>memory_*</code> , <code>map_*</code> , <code>browser_*</code> , <code>plaid_*</code> , etc.
READ_ONLY_TOOLS	<code>file_read</code> , <code>list_dir</code> , <code>grep</code> , <code>find_files</code> (used by de-generate escalation Level 1)
WRITE_TOOLS	<code>apply_patch</code> , <code>file_edit</code> , <code>file_create</code> , <code>file_write</code> (plus shell write pattern detection)

Shell write detection. `_is_write_call()` classifies `shell` calls as writes if their command matches `SHELL_WRITE_PATTERNS`:

```
r"\b(sed\s+-i|tee\s|patch\s|mv\s|rm\s|cp\s|chmod\s|
  chown\s|mkdir\s|touch\s)"
r"|\[^\]|>(?!&)"      # redirect (not pipe)
r"|\>>"              # append
```

6.4 Budget and Limits

[†]`MAX_ITERATIONS` is 100 in the code constant (`loop.py:45`), but the CLI `--max-iters` flag defaults to 50 (`_main__.py:213`). Agents running without an explicit `--max-iters` argument will hit the 50-iteration limit, not 100.

6.4.1 Workspace Calculation

```
workspace = max(soft_limit - initial_tokens, 50,000)
compaction_threshold = [workspace × 0.40]
observe_limit = min([workspace × 0.50], 250,000)
```

6.4.2 Effort-to-Budget Mapping

From `mesh/harness/_main__.py`. This maps the `cc_effort` string to `anthropic_thinking_budget` (thinking tokens for extended reasoning), *not* to the context soft limit:

The context `soft_limit` is independently configurable via `harness_soft_limit` (default: 500,000).

Constant	Default
MAX_ITERATIONS	100 [†]
DEFAULT_SOFT_LIMIT	500,000 tokens
KEEP_RECENT_RESULTS	4
FORCED_SYNTHESIS_FRACTION	0.97
LLM_RETRIES	4
LLM_RETRY_BACKOFF	(2, 4, 8, 16) seconds
TRANSIENT_HTTP_CODES	{400, 429, 500, 502, 503, 504}
CHECKPOINT_STALL_TURNS	3
DEGENERATE_ESCALATION_LIMIT	3
DEFAULT_MAX_PHASES	15
DEFAULT_COMPACTION_THRESHOLD_FRACTION	0.40
ASSESSMENT_MAX_RETRIES	2
PLAN_MAX_RETRIES	2
HALLUCINATION_MAX_RETRIES	2
WATCHDOG_MAX_CONTEXT_TOKENS	100,000
CCWatchdog.RECENT_WINDOW	16

Effort	anthropic_thinking_budget (tokens)
low	2,000
medium	5,000
high	10,000
xhigh	50,000

6.5 Configuration Knobs

All P&E-relevant configuration flows through CLI arguments in `mesh/harness/_main_.py` and the `LLMBackendConfig` class in `mesh/config.py`:

Checkpoint tuning knobs (per-call, not config):

Config Field	Type	Default
<code>harness_controller_mode</code>	str	"standard"
<code>harness_soft_limit</code>	int	500,000
<code>harness_max_phases</code>	int	15
<code>harness_compaction_threshold_fraction</code>	float	0.40
<code>harness_backend</code>	str	(varies)
<code>harness_toolset</code>	str	"legacy"
<code>harness_assessor_model</code>	str	(varies)
<code>harness_assessor_backend</code>	str	(varies)
<code>thinking_budget*</code>	int	(none)
<code>harness_assessor_effort</code>	str	(none)
<code>harness_codex_assessor</code>	bool	false
<code>harness_codex_assessor_model</code>	str	(none)
<code>harness_codex_assessor_effort</code>	str	(none)
<code>harness_codex_assessor_binary</code>	str	"~/local/bin/codex"

Table 6.1: P&E configuration fields on `LLMBackendConfig` (`mesh/config.py`). `*thinking_budget` lives on `LLMConfig` (`mesh/llm.py`), not `LLMBackendConfig`; it is wired through via `backend_config_to_llm_config()`. In production, `harness_assessor_effort: xhigh` is set only on `mesh-harness-qwen36-codex-pe`, not on all P&E backends.

Parameter	<code>run_loop()</code> arg	P&E default
Periodic interval	<code>checkpoint_periodic_interval</code>	5
Periodic start	<code>checkpoint_periodic_start</code>	8
Stall turns	<code>CHECKPOINT_STALL_TURNS</code>	3 (hardcoded)
Escalation limit	<code>DEGENERATE_ESCALATION_LIMIT</code>	3 (hardcoded)

Chapter 7

Plan-and-Execute Controller

The Plan-and-Execute (P&E) controller is a multi-phase execution controller that wraps the standard TAOR loop with an outer assessor controller. At each phase boundary, the assessor chooses one of four intents—**observe**, **plan**, **execute**, or **terminate**.

The controller exists to decompose complex tasks (code edits, bug fixes, document production) into sequential phases, each with a focused objective and bounded context. A separate assessor LLM (typically DeepSeek-v4-pro) drives phase transitions, while the primary LLM (the executor) performs the actual work within each phase.

Entry point. The P&E controller is activated when `run_loop()` is called with `controller_mode="plan_and_execute"`. It immediately delegates to `_run_plan_and_execute_loop()`, which runs the outer control loop.

7.1 Phase Flow

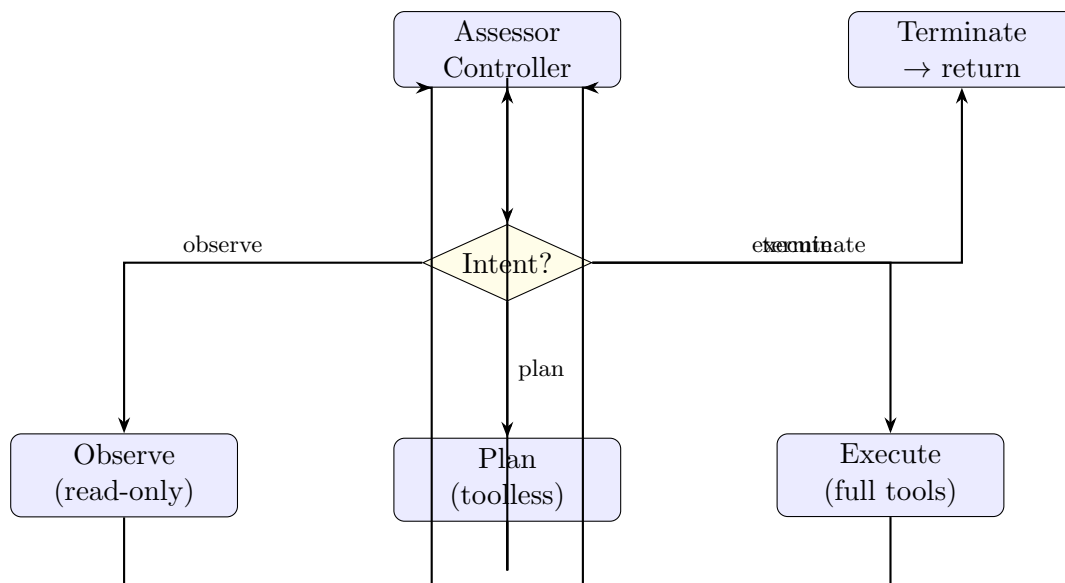


Figure 7.1: P&E controller phase flow. Each non-terminal phase loops back to the assessor controller, which selects the next intent.

The controller iterates up to `DEFAULT_MAX_PHASES = 15`. At each iteration, it:

1. Builds an evaluator context XML document via `_build_evaluator_context()`.
2. Appends an `<action_history>` section summarizing all prior phases.
3. Calls the assessor LLM with the controller template and `max_iterations=1` (no tools, single turn).
4. Parses the response via `parse_controller_intent()` into a `ControllerIntent` dataclass.
5. Dispatches to the appropriate phase handler.

Default on parse failure. If the intent tag cannot be parsed, `parse_controller_intent()` defaults to `observe`—the safest option since it is read-only.

Phase cap enforcement. If the loop exhausts all `max_phases` iterations without terminating, a hard error message is returned.

7.1.1 Phase Tool Exposure

Phase	LLM	Available Tools
Controller (assessor)	Assessor (e.g., DS-pro)	<code>tool_names=[]</code> — no tools; emits XML intent only
Observe	Executor backend	<code>PLANNER_TOOLS</code> $\cup$ (agent's <code>tool_names</code> $\cap$ <code>MESH_READ_ONLY_TOOLS</code>)
Plan	Controller LLM	<code>tool_names=[]</code> — single call, parses plan XML
Execute	Executor backend	Full agent <code>tool_names</code> (all configured tools + <code>HARNESS_TOOLS</code>)

Table 7.1: Tool exposure per plan-and-execute phase. The observe phase cannot trigger write or confirmation-gated tools.

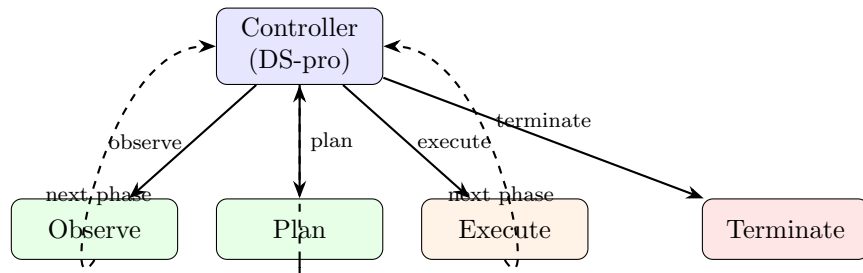


Figure 7.2: Controller loop. Each phase feeds back to the controller for the next intent decision. Terminate exits the loop and returns the final response.

7.2 Assessor Controller

The assessor is a separate LLM client (`assessor_llm_client`), defaulting to the primary `llm_client` if not provided. In production, this is typically DeepSeek-v4-pro.

7.2.1 Controller Invocation

The controller is invoked as a *single-turn, toolless* `run_loop()` call:

```
controller_output = await run_loop(
    llm_client=_assessor,
    history=[HistoryMessage(
        from_node="user",
        content=eval_ctx,      # evaluator context XML
        ...)],
    tool_names=[],            # no tools
    max_iterations=1,         # single turn
    instructions=phase_instructions, # controller template
    ...
)
```

The controller sees a fresh single-message history containing only the evaluator context—it does not share the executor’s conversation history.

7.2.2 Controller Template

The template (`_CONTROLLER_TEMPLATE`) presents four intents with XML output format:

- `<intent>observe</intent> + <prompt>`
- `<intent>plan</intent> + <plan>`
- `<intent>execute</intent> + <prompt>`
- `<intent>terminate</intent> + <summary>`

7.2.3 Evaluator Context

The function `_build_evaluator_context()` constructs an XML document:

```
<evaluator_context>
  <task>...</task>
  <plan>...</plan>                                <!-- current plan, if any -->
  <prior_phases>                                  <!-- compacted phase summaries -->
    <phase_summary phase="N">...</phase_summary>
  </prior_phases>
  <current_phase phase="N">
    <objective>...</objective>
    <tool_history>                                <!-- faithful tool call/result copies -->
      <tool_call name="...">...</tool_call>
    </tool_history>
    <files_modified>...</files_modified>
  </current_phase>
  <action_history>                                <!-- appended by P&E loop -->
    - Phase 1 (observe): ...
    - Phase 2 (plan): ...
  </action_history>
</evaluator_context>
```

Prior phase summaries are populated either from an explicit list or by scanning history messages for `<phase_summary>` tags. Files modified are tracked by matching `WRITE_TOOLS` tool names and

extracting file paths via regex.

7.3 Triage

Note: Triage is *not* currently invoked by the P&E controller loop. `_TRIAGE_TEMPLATE` and `parse.triage()` exist in the codebase but the P&E loop does not call them—the first phase simply goes to the controller for an intent decision. The controller template’s guidelines (“If the task is a simple factual question you can answer now, terminate immediately”) serve the same purpose implicitly.

Triage classifies tasks into three categories:

1. **Done** — answerable from knowledge, no tools needed.
2. **Search-only** (`needs_plan=false`) — pure information retrieval; findings are the deliverable.
3. **Needs plan** (`needs_plan=true`) — requires producing an artifact (code, documents, etc.).

The `TriageDecision` dataclass captures the parse result. On parse failure, `parse.triage()` defaults to `needs_plan=True`.

7.4 Observe Phase

When the controller selects `observe`, the executor runs a read-only exploration loop:

- **Tools:** `search_tools = PLANNER_TOOLS` (`file.read`, `list_dir`, `grep`, `find_files`) plus any `MESH_READ_ONLY_TOOLS` found in the task’s `tool_names`.
- **Soft limit:** `observe_limit = min(⌊workspace × 0.50⌋, 250,000)`.
- **Checkpoints:** enabled, with `checkpoint_periodic_interval=5`, `checkpoint_periodic_start=8`.
- **Instructions:** `OBSERVE_INSTRUCTIONS` appended to the controller’s prompt.

No compaction after observe. History from observe phases is preserved intact so that subsequent plan/execute phases can see the raw tool results (commit `d8d74ca5`).

Phase discipline is advisory, not enforced. The observe phase uses `PLANNER_TOOLS` (read-only) but this is enforced only at the tool-set level. If the executor’s backend (e.g., Claude Code) has its own tools, those are not restricted by the harness tool-name filter.

7.5 Plan Phase

When the controller selects `plan`, the plan is extracted inline from the controller’s own output—no separate LLM call is made. The plan is stored in `current_plan` and injected into subsequent executor phases via `_EXECUTOR_FRAMING` and into assessments via `_ASSESSMENT_TEMPLATE`.

Plan revision. The controller can revise the plan at any time by selecting `plan` again. Additionally, the legacy assessment path supports `<revised_plan>` tags that replace `current_plan`.

Relationship to legacy plan phase. The `PLAN_INSTRUCTIONS` template and `extract_plan_and_phase_prompt()` are remnants of the earlier fixed-sequence controller (observe → plan → execute). In the current assessor-controller design, the plan phase is handled inline by the controller itself, and these functions are not called by the P&E loop.

7.6 Execute Phase

When the controller selects `execute`, a full-power TAOR loop runs:

- **Tools:** the original `tool_names` passed to `run_loop()` (typically `HARNESS_TOOLS` plus mesh read/write tools).
- **Soft limit:** the full `soft_limit` (not the reduced observe limit).
- **Instructions:** `_EXECUTOR_FRAMING` template wraps the controller’s prompt with plan context.
- **Checkpoints:** enabled, `periodic_interval=5`, `periodic_start=8`.
- **CCWatchdog:** enabled for `claude-code` and `zai` backends when `assessor_llm_client` is provided.

7.6.1 Executor Framing

The executor receives its instructions via `_EXECUTOR_FRAMING`, which injects the current plan and phase-specific goal into a structured template. The framing tells the executor to focus only on the current phase’s goal and not to start the next phase.

7.7 Observe Cap

After 2 observe phases (lifetime, not per-segment), the observe intent is disabled. This is a two-layer enforcement (commit `bb62ff3c`):

7.7.1 Layer 1: Template Swap

When `observe_count >= 2`, the controller template is replaced with `_CONTROLLER_TEMPLATE_NO_OBSERVE` via `_strip_observe_from_instructions()`. The no-observe template begins with:

***observe is temporarily disabled** — you have already used your observation budget.
Choose plan, execute, or terminate.*

If the template swap fails (template not found in instructions), a `RuntimeError` is raised as a drift-protection mechanism.

7.7.2 Layer 2: Hard Enforcement

If the assessor LLM ignores the template and returns `observe` anyway, the phase is *skipped* (not force-terminated). A warning is logged and the action history records the skip.

Lifetime cap. The `observe_count` is incremented on each observe phase and is *never reset*—not even after execute phases. The cap is lifetime, not per-segment.

7.8 Safety Properties

- **Observe cap:** After 2 observe phases, the observe intent is permanently removed from the controller’s option set for the remainder of the task.
- **Observe checkpoints:** Every 5 iterations starting at iteration 8, the assessor is asked whether to continue or complete the phase.
- **History carryover:** Observe-phase tool results (file reads, searches) are preserved in the shared `history` list for subsequent phases. No compaction fires between observe and execute—only between consecutive execute phases.
- **Read-only observe:** The `MESH_READ_ONLY_TOOLS` whitelist ensures the observe phase cannot send emails, create events, or perform other write operations.

7.9 Key Commits

54a33f8a

feat: plan-and-execute controller, codex backend, assessor streaming, greenfield benchmarks. Initial P&E implementation with fixed observe $\rightarrow$ plan $\rightarrow$ execute sequence and assessor-driven phase transitions.

c4bc1763

feat: flexible assessor controller, synthesis/compaction dead-zone fix, gpt-oss reasoning fallback. Replaced fixed sequence with flexible four-intent controller (observe/plan/execute/terminate).

bb62ff3c

fix: P&E controller — lifetime observe cap, skip-not-force semantics, helpers. Changed observe cap from per-segment to lifetime.

d8d74ca5

fix: preserve observe-phase history for executor — stop compacting between phases.

177d8689

feat(harness): per-turn context budget meter + forced-synthesis cutoff. Added continuous budget awareness line and the forced synthesis mechanism.

Chapter 8

Native Multi-Turn Executor Loop

The XML-serialized history path (`run_loop`) has a fundamental deficiency for multi-phase execution: tool results from prior phases appear to the executor as `<message from="system">` XML text rather than native API tool results. The executor model treats them as someone else's notes and re-reads the same files, duplicating context and accelerating compaction.

The **native executor loop** (`run_native_loop`) solves this by maintaining the conversation as a native OpenAI messages: `list[dict]` instead of `list[HistoryMessage]`. Tool results are stored as `{"role": "tool", "tool_call_id": "..."} messages—the executor treats them as its own prior work and builds on them across phase boundaries.`

8.1 Location

`mesh/harness/loop.py`

`run_native_loop()`, plus helper functions `estimate_native_tokens()`, `_tool_name_for_call_id()`, `_build_evaluator_context_from_native()`, `_compact_native_history()`.

`mesh/llm.py`

`complete_multi_turn()`—OpenAI Chat Completions call that accepts pre-built messages and skips XML serialization entirely.

`mesh/tools.py`

`ToolCall.call_id` field—preserves the API-returned tool call ID for native message linking.

8.2 Activation

Native mode activates when `backend == "openai"` inside `_run_plan_and_execute_loop()`. Non-OpenAI backends (CC, zai, mesh-harness XML) fall back to the existing XML path. The dispatch is purely conditional—no configuration flag required.

8.3 Message Format

The conversation uses standard OpenAI Chat Completions format:

```
# System + task (protected zone, never compacted)
{"role": "system", "content": "<system_prompt + tool_prompt>"}
{"role": "user", "content": "<task + instructions>"}
```



```
# Phase framing (injected by P&E controller before each phase)
{"role": "user", "content": "<observe/execute instructions>"}

# Assistant turn with tool calls
{"role": "assistant", "tool_calls": [
  {"id": "call_abc", "type": "function",
   "function": {"name": "file_read", "arguments": "..."}}
]}
# Tool result linked by call_id
{"role": "tool", "tool_call_id": "call_abc",
 "content": "<file contents>"}
```

8.4 Cross-Phase Carry-Forward

The `native_messages` list is shared across observe and execute phases via the P&E controller. When the assessor transitions from observe to execute:

1. Observe phase tool results remain in `native_messages` as native `role=tool` messages.
2. The P&E controller appends the executor framing as a `role=user` message.
3. The execute phase calls `run_native_loop()` with the same `native_messages` list.
4. The executor LLM sees all observe-phase file reads as its own prior tool results and does not re-read them.

8.5 Per-Iteration Budget Injection

On each iteration, `run_native_loop` appends a transient `role=user` message containing the budget line and any per-iteration instructions. This message is removed after the LLM responds so it does not accumulate in the conversation history. During forced synthesis, the message stays (it is the final instruction to produce output).

8.6 Assessor Boundary

The assessor remains on the XML path. When the P&E controller needs a phase decision:

1. `_build_evaluator_context_from_native()` converts the native messages into structured XML (`<evaluator_context>`).
2. The assessor call uses `run_loop` (XML mode) with a single-message history containing the evaluator context.
3. The assessor never sees native messages directly.

This separation is intentional: the assessor needs a structured bird's-eye view for phase decisions, while the executor needs native tool context for implementation work.

8.7 Compaction in Native Mode

When scratchpad tokens exceed `compaction_threshold` after an execute phase:

1. `_compact_native_history()` is called with `compact_after` (protected zone boundary).
2. A split-safety guard walks backward from `compact_after` to ensure it does not bisect an assistant→tool_result group (would create orphaned tool_calls or dangling tool results).
3. Compactable messages are serialized to text and sent to the LLM via `complete_with_tools` (XML path) for summarization.
4. The compactable region is replaced by a single `<phase_summary>` user message.
5. `last_native_phase_start_idx` is set to `len(native_messages)` after compaction to avoid stale indices.

8.8 `complete_multi_turn` Method

Located in `mesh/llm.py`, this method:

- Accepts `messages`: `list[dict]` in OpenAI format.
- Passes them verbatim to the Chat Completions API (no XML serialization).
- Accepts pre-converted `tools`: `list[dict]` from `ToolRegistry.get_openai_tools()`.
- Handles reasoning effort negotiation (xhigh→high fallback, non-reasoning model stripping).
- Returns `(content, tool_calls, usage)`.
- `parallel_tool_calls` parameter (default `True`) controls whether the API is allowed to issue multiple tool calls per turn.

8.9 Protocol Events

The native loop emits the same protocol events as the XML loop:

- `emit_tool_call(name, arguments, call_id)` before each tool execution.
- `emit_tool_result(name, result, call_id, success)` after each tool execution.
- `emit_turn_started`, `emit_assistant_message`, `emit_thread_started/finished`, `checkpoint` events.

Chapter 9

Context Management

9.1 Forced Synthesis

Forced synthesis is a budget-exhaustion mechanism in the *inner* TAOR loop (`run_loop()`).

$$\text{synthesis_threshold} = \lfloor \text{soft_limit} \times 0.97 \rfloor$$

At each iteration, `estimate_history_tokens(history)` is compared against the threshold. When exceeded:

1. All tools are stripped (`effective_tool_names = []`).
2. The instructions are replaced with a forced-synthesis message directing the model to produce its final response immediately.
3. A budget line is prepended showing current and maximum token counts.
4. An error event is emitted (non-fatal).

The budget line is injected on *every* iteration, even before forced synthesis triggers, giving the model continuous awareness of context consumption.

9.2 Compaction

9.2.1 When It Fires

Compaction runs *only after execute phases*. It fires when:

$$\text{scratchpad_tokens} = \text{estimated_tokens} - \text{initial_tokens} > \text{compaction_threshold}$$

where:

$$\begin{aligned} \text{workspace} &= \max(\text{soft_limit} - \text{initial_tokens}, 50,000) \\ \text{compaction_threshold} &= \lfloor \text{workspace} \times 0.40 \rfloor \end{aligned}$$

No compaction after observe phases. This was an intentional fix (commit d8d74ca5) to preserve raw observation data for the planner.

9.2.2 Protected Zone

`compact_after` marks the boundary between protected and compactable history. It is set to the initial history length and never moves. Everything before `compact_after` (the original task prompt) is never touched.

9.2.3 Compaction Procedure

The function `_compact_history()`:

1. Concatenates all messages after `compact_after` into a single string.
2. Formats `_COMPACTION_TEMPLATE` with the phase prompt and appends the raw text.
3. Calls the primary LLM (not the assessor) with a fresh single-turn conversation.
4. Extracts content from `<phase_summary>` tags if present.
5. Deletes all messages after `compact_after` and inserts a single summary message.

The compaction template instructs the LLM to preserve five categories: actions attempted, successes and key facts, failures with verbatim error messages, artifacts produced, and open sub-questions.

Failure mode. If the compaction LLM call fails, the first 2000 characters of `executor_output` are preserved as a fallback.

9.3 Degenerate Loop Detection

The checkpoint system detects and responds to degenerate executor behavior (reading the same files repeatedly without progress). It operates as a *side-channel*—checkpoint Q&A is NOT appended to the main conversation history.

9.3.1 Tracker State

Maintained per `run_loop()` invocation:

Variable	Purpose
<code>_ckpt_has_written</code>	Set <code>True</code> on first write tool call
<code>_ckpt_read_only_streak</code>	Consecutive turns with only read tools
<code>_ckpt_last_tool_sig</code>	Sorted tuple of (name, args) from last turn
<code>_read_tools_stripped</code>	Whether read tools are currently removed
<code>_consecutive_degenerate</code>	Count of consecutive degenerate checkpoints

9.3.2 Checkpoint Triggers

Three conditions fire a checkpoint, in priority order:

1. **post_write_stall:** A write has occurred (`_ckpt_has_written`) and `_ckpt_read_only_streak` ≥ 3 .
2. **duplicate_read:** The current tool signature matches the previous turn’s signature exactly.
3. **periodic:** `iteration` $\geq$ `start` and $(\text{iteration} - \text{start}) \% \text{interval} == 0$. Default: `start=8`, `interval=8` (configurable per caller; P&E uses `interval=5`, `start=8`).

9.3.3 Three-Level Escalation

When `degenerate = True`:

Reset conditions.

- `completed = True` resets `_consecutive_degenerate` to 0.
- A write tool call resets `_consecutive_degenerate` to 0 and restores read tools if stripped.
- Neither `completed` nor `degenerate` (both `NO`) resets `_consecutive_degenerate` to 0.

Level	<code>_consecutive_degenerate</code>	Action
1	1	Strip read-only tools (<code>file_read</code> , <code>list_dir</code> , <code>grep</code> , <code>find_files</code>) from subsequent iterations. Restored on next write.
2	2	Strip all <code><reasoning></code> tags from history via regex. Reduces context size by removing chain-of-thought tokens.
3	≥ 3	Force-terminate the phase with an error message.

9.4 CC Watchdog

The `CCWatchdog` class monitors Claude Code (`claude-code`) and `zai` backend executor sessions.

9.4.1 Architecture

Unlike checkpoints (which are fired by the inner TAOR loop), the watchdog is a callback invoked by the CC/zai streaming integration every 8 CC turns. It receives new turn batches and accumulates the full phase history internally.

9.4.2 Evaluation

Each watchdog call:

1. Builds evaluator context XML with a `<recent_activity>` demarcation for the last `RECENT_WINDOW=16` turns.
2. Truncates oldest turn results if context exceeds `WATCHDOG_MAX_CONTEXT_TOKENS = 100,000`.
3. Calls the assessor LLM with `_WATCHDOG_PROMPT` asking YES/NO on progress.

9.4.3 Kill Logic

- **YES** (progress): reset `_consecutive_no` to 0.
- **NO** (stuck): increment `_consecutive_no`.
- **Kill** when `_consecutive_no` ≥ 2 .

On assessor call failure, the watchdog defaults to “continue.”

9.5 In-Flight Context Pruning

`_manage_in_flight_context()` prunes old tool results in the standard TAOR loop when estimated tokens exceed `soft_limit` $\times 0.8$. It keeps the most recent `KEEP_RECENT_RESULTS=4` in-flight messages and inserts a `[N previous tool result(s) omitted]` marker.

9.5.1 Extreme Result Truncation

`_truncate_extreme_result()` caps any single tool result at `soft_limit` $\times 3$ characters.

9.5.2 Tool Result Size Gating

In `_execute_tool_calls()`, individual tool results exceeding `soft_limit × 0.4` characters trigger a size gate (the result is truncated with a marker).

9.6 Codex Assessor

As an alternative to the LLM-based assessor (which uses `complete_with_tools` via the standard API path), the harness supports a **codex CLI subprocess** as the assessor controller. This is implemented in `_run_codex_assessor()` in `mesh/harness/loop.py`.

Motivation. The codex CLI provides built-in reasoning (via OpenAI’s o3 or similar models) with a simpler interface—no tool schema management or XML parsing required. It was introduced as a second attempt after a clean revert (commit 54a33f8a) of the original implementation.

Invocation. The function builds a subprocess command:

```
codex exec --model <model> -s read-only --ephemeral --json \
    --config "reasoning_effort=<effort>" -C <cwd> -
```

Flags: `--ephemeral` (no persistent state), `--json` (JSONL output), `-s read-only` (sandbox), and `-` (read prompt from stdin). The prompt is the concatenation of the evaluator context XML and the controller instructions template—the same `_CONTROLLER_TEMPLATE` used by the LLM assessor.

Execution. The subprocess is launched via `asyncio.get_running_loop().run_in_executor(None, ...)` to avoid blocking the event loop, wrapped in `asyncio.wait_for` with a `CODEX_ASSESSOR_TIMEOUT` of 300 seconds (plus 10s grace for the outer wait).

Output parsing. The codex CLI with `--json` emits JSONL events. The parser looks for two event patterns:

1. Events with `type=message`, `role=assistant`: extracts the `output_text` content part.
2. Events with `type=response.completed`: extracts assistant message content from the nested `response.output` array.

If neither yields structured content, the parser falls back to raw stdout—checking for `<intent>` tags directly in the output.

Failure handling. On timeout or non-zero exit code, the function returns a synthetic `<intent>terminate</intent>` response with an error summary. This ensures the P&E controller always receives a parseable intent, even on infrastructure failure.

Configuration. Three fields on `LLMBackendConfig` control the codex assessor: `harness_codex_assessor` (bool, enables it), `harness_codex_assessor_model` (default: "o3"), `harness_codex_assessor_effort` (reasoning effort string), and `harness_codex_assessor_binary` (path to the codex CLI binary, default: `"~/local/bin/codex"`).

9.7 Known Discrepancies and Issues

1. **Phase discipline is advisory.** The observe phase restricts tools via `tool_names` but CC/zai backends have their own tool surfaces. The harness cannot prevent a CC executor from editing files during an “observe” phase.
2. **Assessor token overhead.** Operational data shows the assessor consuming up to 65.7% of total token budget. Each phase transition requires a full evaluator context XML document.
3. **Assessor hallucination.** The assessor (a toolless LLM call) frequently attempts to emit tool calls. The harness retries up to `HALLUCINATION_MAX_RETRIES=2` times.
4. **Triage is dead code in P&E path.** `_TRIAGE_TEMPLATE`, `parse_triage()`, and `TriageDecision` exist but are not called by `_run_plan_and_execute_loop()`.
5. **Legacy plan infrastructure.** `PLAN_INSTRUCTIONS`, `extract_plan_and_phase_prompt()`, and `_ASSESSMENT_TEMPLATE` are remnants of the fixed-sequence controller and are not invoked.
6. **Compaction uses primary LLM, not assessor.** `_compact_history()` calls the executor’s LLM, not the assessor—potentially using an expensive model for summarization.
7. **400 treated as transient.** `TRANSIENT_HTTP_CODES` includes 400 (Bad Request), which is typically a permanent client error, not transient.

Part III

Memory and Knowledge

Chapter 10

Memory System

The memory system gives agents persistent, project-organized episodic memory that survives across sessions. It operates at three layers: conversation history (short-term), episodic memory pool (medium-term), and project maps (structural).

10.1 Architecture Overview

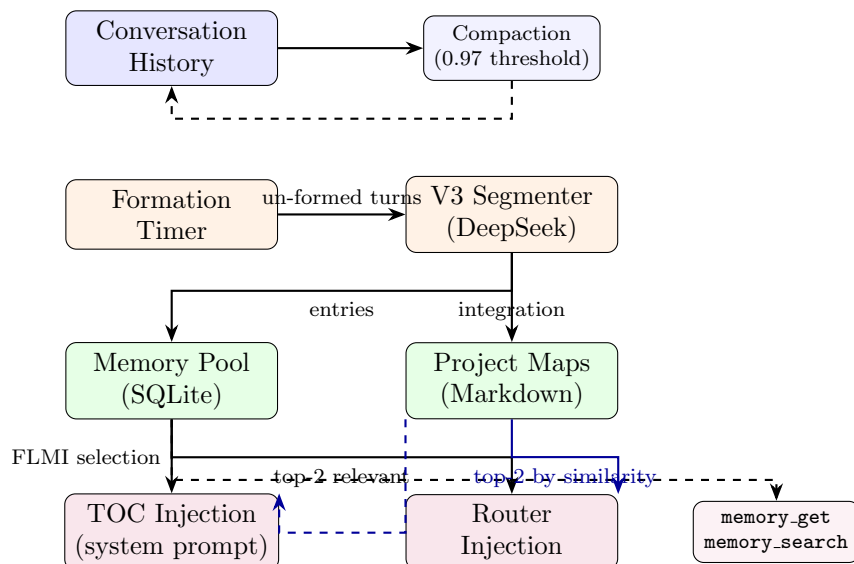


Figure 10.1: Memory system data flow. The formation timer drives the V3 segmenter, which produces entries persisted to the memory pool and integrated into project maps. Retrieval surfaces memories via TOC injection (always-on), router injection (per-dispatch), relevance-based map injection (blue arrows, embedding similarity against recent turns), and on-demand tools (`memory_get`, `memory_search`).

Three layers.

Conversation history The raw message thread between user and agent. The router compacts it as context grows, using a forced synthesis threshold of 0.97 of the soft context limit. Recent

messages stay intact; older turns are summarized. This provides short-term recall within a session.

Memory pool Episodic memories stored in a SQLite database (`~/mesh/memory/{agent}.db`). Each entry has three tiers of detail (Table ??). The pool persists across sessions and grows over the agent’s lifetime.

Project maps Living Markdown summaries of each project’s state—goals, architecture, key files, decisions, current status, and open issues. Stored centrally in `~/mesh/memory/maps/{project_name}.md` (constant `MAPS_DIR` in `mesh/paths.py`) and indexed in the `project_maps` SQLite table.

Tier	Name	Content
1	Summary	Short retrieval key (1–2 sentences). Always visible in the TOC.
2	Reflection	Detailed analysis: what happened, why it matters, lessons learned. Fetched via <code>memory_get(id)</code> .
3	Trace	Full tool call trace from the original conversation. Fetched via <code>memory_get(id, full=true)</code> .

Table 10.1: Three-tier memory entry structure. Higher tiers are fetched on demand to avoid context bloat.

10.2 Formation V3 Pipeline

Memory formation converts un-formed conversation turns into structured episodic entries. The V3 pipeline uses a dedicated LLM (`memory_llm_backend`, typically `deepseek-direct`) that is decoupled from the agent’s primary worker backend.

Trigger and timing. Formation fires on two triggers:

1. **Timer:** a periodic timer (configurable interval, typically 10–15 minutes) calls `form_unformed()`.
2. **Conversation boundary:** when the context window drops old turns during compaction, the dropped turns are processed via `on_window_drop()`.

Segmenter. The V3 segmenter (`mesh/memory/formation_v3.py`) sends un-formed turns to the DeepSeek LLM with a structured prompt. The LLM returns a JSON array of segments, each containing:

- `topic_label` — concise label for the segment.
- `summary` — 2–3 sentence retrieval key.
- `reflection` — detailed analysis and lessons learned.
- `project` — project assignment. The prompt encourages creating new project labels when no existing project matches; null is reserved only for trivial exchanges.
- `tags` — categorization tags.
- `outcome` — success, partial, or failure.
- `score` — worthiness score (1–10; entries below 4 are filtered as “not worth remembering”).

Cursor mechanics. A cursor index tracks which conversation turns have been processed. Entry persistence and cursor advancement are atomic—if persistence fails, the cursor stays put and the

window is retried on the next trigger. A cursor clamp (commit `a6f597d9`) prevents the cursor from advancing past the actual history length after restarts.

Failure tolerance. A 3-strike system tracks per-window failures. After 3 consecutive failures on the same window (keyed by `(cursor_idx, end_idx)`), the window is skipped and the cursor advances past it.

Cold-start handling. When no project maps exist, the segmenter is prompted to *suggest* project labels rather than requiring exact matches. The first entry for a new project triggers `_create_new_project_map()`, which bootstraps a skeleton map via a separate LLM call.

10.3 Entry-Driven Map Curation

After the V3 segmenter persists entries, a fire-and-forget `asyncio.create_task` integrates the new entries into their respective project maps.

Flow.

1. `form_un_formed()` persists entries and advances cursor.
2. New-project bootstrap creates skeleton maps for unknown projects.
3. `_integrate_entries_into_maps(entries)` is spawned as a background task:
 - (a) Group entries by `entry.project`, filtering empty projects.
 - (b) Per project: load current map, format entries with date, topic, score, outcome, tags, and summary.
 - (c) Send to DeepSeek LLM (`max_tokens=32768`, 120s timeout) with the integration prompt.
 - (d) LLM produces `<section>` (full section replacement) and `<append.to>` (append to section end) edit operations.
 - (e) `_apply_section_updates()` parses and applies edits.

Design properties.

- **Non-blocking:** integration failures are logged but do not affect entry persistence or cursor advancement.
- **Multi-project:** a single formation batch may span multiple projects; each gets an independent integration call.
- **Composable with bootstrap:** new-project skeleton maps are created before integration runs, so the integrator always has a map to patch.

Canonical section schema. All three map prompts (curation, bootstrap, integration) reference the same canonical section list:

1. **Summary** — 2–3 sentences: what the project is, current phase.
2. **Goals** — what we are trying to accomplish, success criteria, guiding constraints.
3. **Architecture** — system structure, data flow, key abstractions.
4. **Key Files** — navigational map with file paths, grouped by subsystem.
5. **Key Decisions** — decisions made, rationale, dates.
6. **Current State** — what is active, completed, blocked, next.
7. **Open Issues** — unresolved problems, open questions.
8. **Glossary** — abbreviations, project-specific vocabulary.

10.4 Retrieval

Four retrieval paths surface memories and project context to the agent at different points in the processing pipeline.

TOC Injection (Always-On). The top 30 memory entries, ranked by relevance to the current conversation, are injected as a `<memory_toc>` block in the agent’s system prompt. Each entry shows its retrieval key (summary) and ID. Entries already fetched this session are marked `[already in context]`; entries injected into the current worker dispatch are marked `[injected into worker]`.

On-Demand Tools.

- `memory_get(id)` — fetches the full reflection (Tier 2) of a specific entry. The result persists in conversation history for the rest of the session.
- `memory_search(query, project=...)` — semantic search over the memory pool using embedding similarity. Returns top- k entries.

Router-Level Injection. Before dispatching work to a worker, the router runs `_select_memory_context()` to curate and inject relevant memories into the worker’s context. The top 2 entries with a relevance score above 0.3 are formatted as `<router_injected_context>` XML and prepended to the worker’s prompt.

10.5 Feature Flag Matrix

Memory features are deployed incrementally per agent via configuration flags in `mesh.yaml`. Formation V3 uses a dedicated LLM backend (`memory_llm_backend`, typically DeepSeek) decoupled from the agent’s worker backend. All deployed agents use formation V3 with the retrieval redesign.

10.6 Standing Digest (Fold System)

The standing digest extends the memory system with a dense, continuously updated summary document per agent. Where the episodic memory pool stores individual entries with three-tier retrieval, the standing digest is a single coherent narrative—a “standing briefing”—that captures the agent’s full operational history, key decisions, project context, and relationship knowledge in one document.

Architecture. The fold driver (`fold_driver/`) runs as an offline batch process, separate from the live agent. It iterates over the agent’s raw conversation history in chronological chunks (“rounds”), feeding each chunk plus the current digest into an LLM that produces an updated digest. The process is idempotent: a cursor tracks the last-processed history line, so the fold can be interrupted and resumed.

Configuration. Three fields on `NodeConfig` control the fold:

- `fold_backend` (str, default: `"deepseek-direct"`) — the LLM backend used for fold completions.
- `standing_digest_enabled` (bool) — when true, the published digest replaces the `<memory_toc>` block in the agent’s system prompt.
- `standing_digest_path` (str) — path to the published digest file (Markdown).

Quality gates. Each fold round passes through validation before the cursor advances: citation provenance checks (every [m...] reference must resolve to a real memory entry), structural integrity (required sections present), coverage verification (the digest must not drop previously covered material), and placeholder rejection (no unresolved [...] tokens may survive handle expansion).

Relationship to episodic memory. The standing digest and the episodic memory pool are complementary, not competing. The pool stores granular entries for on-demand retrieval; the digest provides always-on holistic context. When `standing_digest_enabled` is true, the digest replaces the TOC injection (which would otherwise list individual entry summaries), while the pool’s search and retrieval tools remain available.

10.7 Key Files

File	Purpose
<code>mesh/memory/system.v2.py</code>	Core memory system: formation, curation, retrieval, FLMI selection, map management, relevance-based injection (4000 lines).
<code>mesh/memory/formation.v3.py</code>	V3 segmenter: JSON-structured LLM segmentation with validation, clamping, and retry logic.
<code>mesh/memory/store.py</code>	SQLite persistence: <code>MemoryEntry</code> dataclass, CRUD operations, embedding storage, project map storage.
<code>mesh/memory/selection.py</code>	FLMI active set selection using embedding vectors.
<code>mesh/memory/embeddings.py</code>	Embedding client for memory entry vectorization.
<code>mesh/paths.py</code>	Central path resolution: <code>MAPS_DIR</code> (<code>~/.mesh/memory/maps/</code>), <code>MEMORY_DIR</code> , real-home bypass for CC agents.
<code>mesh/prompts/memory.md</code>	Shared prompt instructions for memory tools and TOC usage. Auto-appended to all agent prompts via <code>load_prompt_file()</code> .

Table 10.2: Memory system source files.

Chapter 11

Project Maps and TOC Pipeline

11.1 Relevance-Based Map Injection

Project maps are injected into both the router and worker prompts based on relevance to the current conversation, replacing the earlier approach of injecting only the statically-tracked `_active_project` map.

Algorithm.

1. Concatenate the text of the last N conversation turns ($N=5$ by default, hardcoded in the router's `_get_last_n_turns_text()` helper).
2. Embed the concatenated text using the memory embedding client (`text-embedding-3-small`).
3. Compute cosine similarity between the context embedding and each project map's summary embedding (stored in the `project_maps` table).
4. Return the top- k maps ($k=2$) with similarity above a minimum threshold (0.2).

Injection points.

- **Router prompt:** `_build_system_prompt()` and `_build_router_prompt()` call `render_relevant_maps_block()` (which wraps `select_relevant_maps()`), injecting up to 2 maps as `<project_map project="..." relevance="...">` blocks.
- **Worker prompt:** `_select_memory_context()` appends relevant maps to the `<router_injected_context>` block alongside memory entries. Workers previously received no project maps.
- **Fallback:** if no map embeddings are available (embedder down, or maps lack summaries), falls back to `render_maps_block()` (active project only).

Summary embeddings. Each project map maintains a 1–2 sentence summary in the `project_maps.summary` column, extracted deterministically from the map's `## Summary` section after every content change. The summary is embedded via `text-embedding-3-small` and stored in the `embedding` BLOB column. Summaries are better embedding targets than full map content: they capture the project's domain and focus in a dense, semantically rich form, whereas full maps contain structural/list content that dilutes embedding quality.

Design rationale. The static `_active_project` approach had two limitations: (1) only one map was ever injected, even when conversations spanned multiple projects, and (2) workers received no project context at all. Relevance-based selection aligns map injection with the same principle used for memory entry retrieval.

11.2 TOC Selection and Weighting (FLMI)

The active set (top 30 entries shown in the prompt TOC) is selected from the full pool using a two-stage pipeline that balances breadth with diversity.

Stage 1: Cosine relevance. The top 100 entries by cosine similarity to the last N conversation turns are selected as candidates. This ensures the TOC is grounded in the current conversational context.

Stage 2: Facility Location diversity. From the top-100 candidates, the `lazy_greedy_fl()` function (in `mesh/memory/selection.py`) selects 30 entries using a submodular Facility Location objective. This maximizes embedding diversity—ensuring broad coverage of the agent’s topic history rather than clustering around a single recent topic.

Known issue: weight starvation. On agents with large pools, FLMI’s diversity optimization means new entries may receive weight 0 and never enter the active set. The existing 30 entries already cover the topic space, so new entries (which often overlap with recent work) offer no diversity gain. This effectively fossilizes the active set.

Summary maintenance. Each project map has a short summary (1–2 sentences) stored alongside an embedding vector in the `project_maps` table. Summaries are extracted deterministically after every map content change—via integration, tool edits, or manual curation—by parsing the canonical `## Summary` section from the map content (no LLM call needed). If no `## Summary` section exists, the first non-heading paragraph (up to 500 characters) is used as a fallback. The embedding is recomputed from the updated summary and cached in memory. The update runs as a fire-and-forget `asyncio.create_task`.

Part IV

Infrastructure and Operations

Chapter 12

Multi-Host Deployment

The mesh system supports multi-host deployments where a central router coordinates agents running on separate machines. This chapter documents the deployment topology patterns, service management, and cross-host coordination mechanisms. See also `docs/two-host-demo.md` for a concrete walkthrough.

Typical topology. A production deployment assigns distinct roles to hosts:

Primary host Runs the router (optionally systemd-managed), multiple agents, an nginx reverse proxy with TLS termination, and the web client.

Secondary host(s) Run additional agents. Connected to the primary via WebSocket (optionally through an nginx proxy for TLS).

Backup target (optional) Receives daily incremental rsync snapshots from the primary host.

Service management. The router and critical agents can be managed via systemd units with automatic restart on failure. Other agents are managed via `agent-ctl.sh`, a shell script providing start/stop/status operations with backend selection.

Cross-host sync. Syncthing or similar tools can synchronize `~/.mesh` and shared directories across hosts. This ensures memory databases and configuration changes propagate automatically. The application code can be synchronized via git.

Backup. A daily incremental rsync job can copy the mesh state to a backup target with configurable retention.

Chapter 13

Security Hardening

Several hardening measures protect subprocess spawning, credential handling, and inter-process communication.

13.1 Subprocess Environment Allowlist

All LLM subprocess methods (`_run_cc_subprocess`, `_complete_codex`, `_complete_mesh_harness`, `cc_session.py`, `router_cc.py`, `eval/evaluator.py`) filter `os.environ` through `_build_subprocess_env()`, a module-level helper in `mesh/llm.py`. The allowlist is a `frozenset` of 30 approved environment variables plus prefix matching for `CLAUDE_*` and `ANTHROPIC_*`.

Category	Allowed Variables
System	PATH, HOME, USER, SHELL, LANG, LC_*, TERM, TMPDIR, XDG_*, SSH_AUTH_SOCK
API keys	OPENAI_API_KEY, ANTHROPIC_API_KEY, DEEPSEEK_API_KEY, EXA_API_KEY, ZAI_API_KEY, OPENROUTER_API_KEY, SYNTHETIC_API_KEY, GOOGLE_API_KEY
Mesh	MESH_AUTH_TOKEN, MESH_NODE_ID, MESH_ATTACHMENT_SECRET
Python	VIRTUAL_ENV, PYTHONDONTWRITEBYTECODE, PYTHONHASHSEED

Table 13.1: Subprocess environment allowlist categories. Variables not in this list (e.g., `LD_PRELOAD`, `LD_LIBRARY_PATH`, `PYTHONPATH`) are blocked.

Operator-controlled overrides (`cc_env`, `env_override` in `LLMConfig`) are applied *after* filtering and may inject arbitrary variables by design (e.g., account-specific `HOME`, custom API keys).

13.2 API Key Masking in Logs

The `_mask_cmd()` helper in `mesh/llm.py` masks sensitive values in subprocess command logs. It handles both two-token (`--api-key VALUE`) and single-token (`--api-key=VALUE`) forms, replacing values with `***` plus the last 4 characters. Only log output is affected; the actual subprocess command is unchanged.

13.3 Socket Path Hardening

Agent tool sockets moved from the world-readable `/tmp/` directory to `~/.mesh/sockets/` with directory mode 0700 (owner-only). Socket names are deterministic (`{sanitized_node_id}.sock`) within the restricted directory—no random suffix needed since only the owning user can list or connect.

13.4 Node ID Propagation

`MESH_NODE_ID` is set via two independent paths:

1. **Process-level:** `os.environ["MESH_NODE_ID"]` set in `AgentNode.__init__` for tool subprocess inheritance.
2. **Per-LLMClient:** `LLMConfig.node_id` injected explicitly into each LLM subprocess environment. Ensures correctness if multiple `LLMClient` instances exist in one process.

13.5 Agent Node Refactoring

The 842-line `_process_with_llm` method was decomposed into a 280-line orchestrator plus six focused helpers:

- `_setup_controller_for_message` — controller initialization and streaming observer.
- `_build_system_prompt_for_llm` — memory, preferences, personality, trace prompt assembly.
- `_build_worker_instructions` — worker instructions and briefing mode setup.
- `_handle_controller_llm_response` — controller decision routing.
- `_dispatch_special_tool_calls` — 10 special tool types (`send_message`, `schedule_wake`, etc.).
- `_process_tool_calls_in_loop` — full tool iteration processing with retry and error handling.

13.6 Confirmation Gate Bypass

Two mechanisms allow tool calls to skip interactive confirmation:

1. **Socket handler** `skip_confirmation=True` (`agent_node.py`): When a worker subprocess calls a confirmation-required tool (e.g., `gmail_send_message`) via the agent socket, the socket handler executes it with `skip_confirmation=True`. The rationale is that the user already authorized the work by dispatching the worker—re-prompting for confirmation on each sub-action would make the worker unusable.
2. **auto_confirm.tools config:** Per-agent YAML configuration that lists tool names which should always skip confirmation for that agent. Used for Alice’s Gmail write tools in production.

Both mechanisms are intentional design decisions, not security bypasses. The trust boundary is at worker dispatch time, not at individual tool invocation time within an authorized worker session.

13.7 IDLE→BUSY Race Fix

In `router_v2.py`, the worker task is now created *before* `self._state` is set to `BUSY`. Since `asyncio` is single-threaded, the task does not execute until the next yield, but the invariant `_state == BUSY` $\implies$ `_worker_task is not None` now holds at every point.

Chapter 14

Benchmark Infrastructure

The mesh includes a benchmark harness for systematic evaluation of backend configurations against a curated test suite. This chapter documents the infrastructure; results are presented in Chapter ??.

Runner. `benchmark/run_sanity_mesh_harness.py` is the main entry point. It accepts a `--mesh-backend` argument specifying which `mesh.yaml` backend to evaluate, plus optional flags for thinking budget, assessor effort, and codex assessor configuration. The runner iterates over a suite of test instances, launching the harness subprocess for each, collecting patches, and running evaluation (typically `pytest` inside a Docker container matching the target repository).

Backend resolution. `benchmark/_mesh_backend.py` defines `ResolvedBackend`, a dataclass that resolves a backend name from `mesh.yaml` into a complete configuration including model, API endpoint, controller mode, soft limit, thinking budget, and assessor configuration.

Test suites. The primary suite is `benchmark/delta_suite.json`, a 15-instance collection covering 10 hello-world instances (drawn from real mesh system bugs) and 5 SWE-bench instances (astropy, django, ansible, etc.). Additional suites exist for research benchmarks and full SWE-bench evaluation.

Evaluation. Each instance includes a test specification. After the harness produces a patch, the runner applies it to a clean checkout inside a Docker container and runs the test suite. Results are recorded as pass/fail counts per instance, along with token usage, wall time, and iteration counts.

Part V

Evaluation

Chapter 15

Benchmark Results and Ablations

This chapter will present empirical results from benchmark evaluations of different backend configurations, including ablations over thinking budgets, assessor effort levels, and controller modes. Key comparisons include:

- **Standard vs. P&E mode** on the 15-instance delta suite, with per-instance breakdowns showing where the assessor adds value and where it regresses.
- **Thinking budget ablation** (Qwen3.6-27B with and without `thinking_budget: 16384`): standard mode improved from 9/15 (60%) to 11/15 (73%) with thinking enabled, while P&E held at 12/15 (80%) with different failure composition.
- **Heterogeneous backend combinations**: local Qwen executor with DS-pro API assessor, GLM-5.1 executor with DS-pro assessor, and monolithic API-only configurations.
- **Wall time and cost analysis**: standard + thinking at 82 min vs. P&E at 220 min for similar accuracy on a free local model.

Chapter 16

Lessons Learned

This chapter documents operational lessons from running the mesh system in production across multiple agents for 5+ months.

- **Memory retrieval adoption:** Despite a well-populated memory pool (450+ entries across agents), active retrieval (`memory_get`, `memory_search`) remained at zero for 24+ hours post-deployment. Root cause: CC-backend agents lack native tool access to memory tools; the CLI bridge is high-friction. Passive channels (TOC injection, router injection) carried the load.
- **Project mis-tagging:** Formation V3’s conservative prompt (“Do NOT invent new identifiers”) caused entries to collapse into a single project. Fixed by encouraging organic project creation and removing the `_active_project` null fallback.
- **P&E overhead:** The assessor consumes up to 65% of total tokens. For simple tasks, standard mode with thinking enabled matches P&E accuracy at 2.7x lower wall time.
- **Agent coordination:** Ack-to-ack loops between agents in channels required explicit suppression rules. Agents defaulted to acknowledging each other’s messages, creating unbounded ping-pong chains.
- **Standing digest vs. TOC:** The episodic memory TOC (always-on entry summaries) scales poorly past several hundred entries. The standing digest fold system (Section ??) resolved this by replacing the entry-level TOC with a single coherent narrative, trading per-entry granularity for holistic context.
- **Harness-backend router dispatch:** When the router itself runs on a harness backend (Claude Code, Codex), the native tool-call dispatch path is unavailable. The unified dispatch contract (`<dispatch_worker>` XML blocks, stripped router tools, single outer iteration) emerged from repeated failures where harness-backend routers attempted to call tools they could not execute.